**Computación en nube y virtualización de redes y servicios (CNVR)**

**Arquitectura de redes y servicios en la nube (ARSN)**

Curso 2026-27

# Trabajo Final: Despliegue automático de una aplicación escalable sobre Kubernetes utilizando los servicios de orquestación de Terraform

Versión del 5 de noviembre de 2026

## 1 - Introducción

El objetivo del trabajo final de la asignatura consiste en desarrollar los scripts, plantillas y ficheros de configuración necesarios para desplegar y configurar de forma automatizada una aplicación web escalable sobre una infraestructura de Kubernetes utilizando Terraform.

El escenario que se deberá desplegar es el representado en la Figura 1. La arquitectura estará compuesta por un clúster de Kubernetes con el CNI de Calico (escenario de la práctica 1.4), en el que se ejecutarán tres *Pods* de servidores web (desplegados a través de un recurso *Deployment*) y un *Pod* de administración, y un escenario complementario desplegado con VNX, que incluirá un servidor de bases de datos y un servidor de *logs* implementados mediante contenedores LXC.

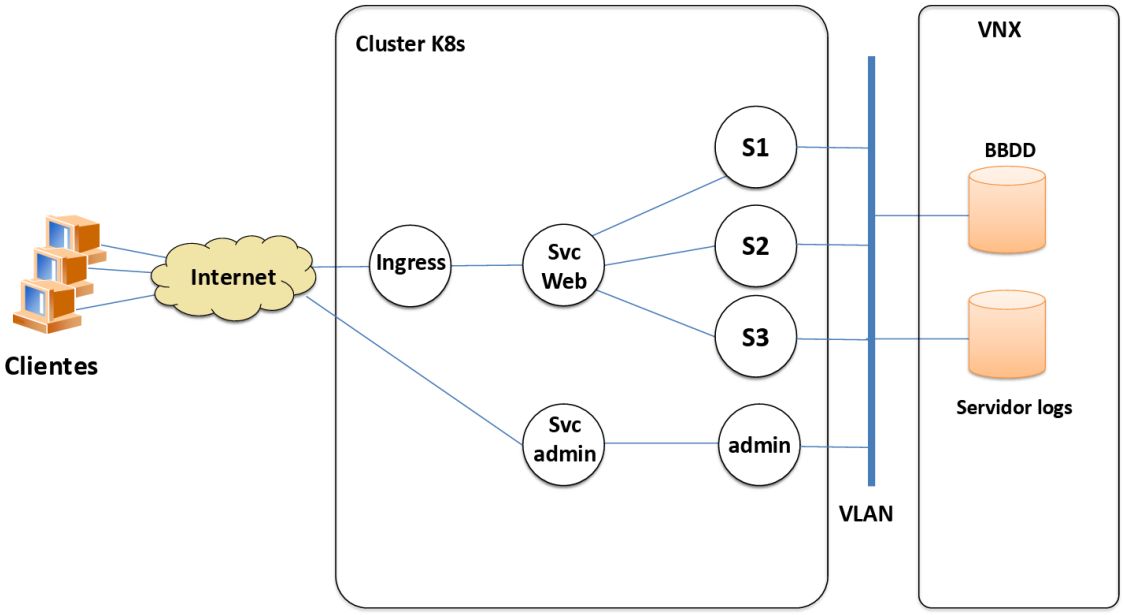



*Figura 1: Topología del servicio a desplegar.*

Los clientes accederán a la aplicación web a través de un recurso *Ingress*, que dirigirá las peticiones al *Service* de Kubernetes encargado de distribuir el tráfico entre los tres *Pods* de servidores web. Estos servidores alojarán el contenido de la aplicación y se configurarán para generar una página web personalizada.

Además, los servidores web, el servidor de administración y los contenedores LXC estarán conectados a una VLAN compartida, utilizando interfaces de red adicionales configuradas mediante Multus en los *Pods* de Kubernetes. Esta red permitirá la comunicación entre los distintos componentes de la arquitectura. El servidor de administración se utilizará para realizar tareas de gestión y para acceder mediante SSH a los servidores web, utilizando autenticación por clave pública/privada.

La infraestructura deberá incorporar los mecanismos de control de tráfico necesarios mediante las políticas de red de Kubernetes (recursos de tipo *NetworkPolicy*), así como la configuración de los *Services*, el *Ingress*, las interfaces de red y los recursos de configuración necesarios para el funcionamiento de la aplicación.

Los requisitos específicos de despliegue, configuración y conectividad de cada uno de estos componentes se detallan en los siguientes apartados.

Para obtener más información sobre los diferentes recursos de Kubernetes a configurar con Terraform, se recomienda consultar la documentación oficial de Terraform para Kubernetes, donde se describe la sintaxis HCL de los recursos disponibles y se incluyen ejemplos de configuración [1]. Además, en la carpeta de ejemplos del repositorio GitHub oficial “Terraform Kubernetes Provider” [2] puede encontrar algunos ejemplos adicionales.

## 2 - Configuración de red VLAN, Multus, contenedores LXC y administración mediante SSH

Se deberá configurar una red VLAN compartida entre los *Pods* de Kubernetes y los contenedores LXC desplegados en un escenario VNX complementario. Esta red permitirá la comunicación entre los servidores web, el servidor de administración, el servidor de bases de datos y el servidor de *logs*. Como orientación, se recomienda seguir los siguientes pasos:

1. **Imágenes de los contenedores para los *Pods* en Kubernetes:** utilizar las imágenes de contenedores que se proporcionan para los servidores web y el servidor de administración, que ya incorporan SSH instalado.
    - Imagen *dgsgiros/admin-ssh:latest* para servidor de administración con usuario *adminuser* y permisos de administrador.
    - Imagen *dgsgiros/nginx-ssh:latest* para servidores web con usuario *webuser* y permisos de administrador.

    Ambas imágenes contienen múltiples herramientas disponibles para realizar pruebas de conectividad y administrar la configuración de red (ping, nmap, curl, ip, ifconfig, etc.). Permiten instalar/actualizar nuevas dependencias con *apt* desde el usuario *adminuser/webuser* usando *sudo*.

2. **Configuración de la VLAN en los nodos *worker* de Kubernetes:** crear y configurar en los nodos worker (*k8s-worker1* y *k8s-worker2*) las subinterfaces de red necesarias para asociar los *Pods* a la VLAN correspondiente, **siguiendo un procedimiento similar realizado en la práctica 1.3**. Indicar que los nodos *worker* están conectados a una red virtual *VlanNet* que será compartida con el escenario de red VNX donde se alojan los contenedores LXC (BBDD y servidor de *logs*) para interconectar todas las máquinas (*Pods* y contenedores LXC) a través de la VLAN definida (**ver paso 6** para saber más sobre la configuración en los contenedores LXC).
3. **Configuración de Multus en Kubernetes:** definir los recursos *NetworkAttachmentDefinition* necesarios para configurar una interfaz de red adicional en los *Pods* de los servidores web y del servidor de administración. Estas interfaces deberán conectarse a la VLAN compartida con los contenedores LXC del escenario VNX. **Seguir un procedimiento similar al realizado en la práctica 1.3.**
4. **Configuración de acceso por SSH a través de red VLAN:** utilizar la red VLAN compartida para que el servidor de administración pueda acceder mediante SSH a las distintas máquinas conectadas a dicha red. La autenticación deberá realizarse mediante un par de claves SSH pública y privada, evitando el uso de contraseñas. Para el acceso por SSH desde el servidor de administración a los *Pods* de los servidores web, así como el acceso por SSH desde fuera del clúster de Kubernetes al servidor de administración, **seguir las recomendaciones del paso 4**.
5. **Configuración de las claves SSH mediante *Secrets*:** crear los recursos *Secret* necesarios para distribuir las claves SSH a los *Pods* y permitir mediante autenticación por clave pública/privada: (1) el acceso desde el servidor de administración a los servidores web; y (2) el acceso desde el exterior del clúster de Kubernetes al servidor de administración.

   Para ello, se recomienda generar previamente en los host dos pares independientes de parejas de claves SSH (pública y privada), uno para cada caso de acceso indicado:

   I. Primer par (administración → servidores web): la clave privada se proporcionará al *Pod* del servidor de administración, mientras que la clave pública se configurará como autorizada en los servidores web.
   II. Segundo par (exterior → administración): la clave privada se conservará en el host desde el que se acceda al clúster, mientras que la clave pública se configurará como autorizada en el *Pod* del servidor de administración.

   A continuación, se indican las recomendaciones para distribuir y configurar estas claves en los diferentes *Pods*:

   - **Servidor de administración:** proporcionar una clave privada SSH que utilizará para conectarse a los servidores web y una clave pública SSH configurada como autorizada (copiada en fichero *authorized_keys* de configuración de SSH del usuario) para

permitir el acceso SSH desde el exterior del clúster. Se recomienda utilizar un recurso *Secret* independiente para copiar cada una de las dos claves.

- **Servidores web:** proporcionar la clave pública SSH ligada a la clave privada SSH del servidor de administración y configurarla como clave autorizada (copiada en fichero *authorized_keys* de configuración de SSH del usuario) para permitir el acceso SSH desde el servidor de administración. Por facilidad, se recomienda usar la misma clave pública para todos los servidores web, y hacer uso de un único recurso *Secret* para copiarla en los *Pods.* Esta misma clave pública puede ser utilizada para acceder a los contenedores LXC (BBDD y Servidor de *logs*), **siguiendo las recomendaciones indicadas en el paso 6**.

Para la creación y utilización de recursos Secret se recomienda consultar la documentación oficial de Kubernetes sobre *Secrets* [4], y en particular el apartado “Caso de Uso: Pod con llaves ssh”, donde se muestra cómo crear directamente mediante *kubectl* un *Secret* a partir de ficheros de claves SSH, sin necesidad de definir previamente el objeto *Secret* en un fichero YAML. Por ejemplo:

```
kubectl create secret generic ssh-private-key \
--from-file=id_rsa=./ssh/id_rsa
```

En este caso, la clave almacenada en el *Secret* con el nombre *id_rsa* puede montarse posteriormente como fichero dentro del *Pod* mediante un volumen. En la especificación del volumen pueden establecerse los permisos de los ficheros mediante las propiedades *defaultMode* y *mode*: *defaultMode* establece los permisos por defecto y *mode* permite establecer permisos específicos para cada fichero.

Por ejemplo:

```
volumes:
  - name: ssh-private-key
    secret:
      secretName: ssh-private-key
      defaultMode: 0440
      items:
        - key: id_rsa
          path: id_rsa
          mode: 0440
```

En el caso del *Pod* de administración, se recomienda utilizar *securityContext.fsGroup* en la especificación del propio *Pod* para asociar los volúmenes montados al grupo del usuario que ejecuta SSH. En este ejercicio, tanto *adminuser* como *webuser* utilizan el GID (identificador de grupo) 1000, por lo que en el *Pod* de administración puede especificarse:

```
spec:
  securityContext:
    fsGroup: 1000
```

De esta forma, el usuario *adminuser* puede acceder a la clave privada montada desde el *Secret* utilizando los permisos de grupo.

Investigar cómo crear y gestionar estos recursos *Secret* directamente mediante Terraform, utilizando los recursos y mecanismos que proporciona su proveedor para Kubernetes, en lugar de recurrir a comandos de *kubectl*.

6. **Configuración de los contenedores LXC en VNX:** configurar los contenedores que actuarán como servidor de bases de datos y servidor de *logs* para que estén conectados a la VLAN compartida. Para el servidor de bases de datos, se propone instalar MariaDB mediante comandos de instalación ejecutados en el contenedor LXC con instrucciones de tipo *exec* de VNX. Estas mismas instrucciones se pueden utilizar para instalar y activar el servicio de acceso por SSH en los contenedores LXC. Asimismo, se utilizarán instrucciones de tipo *filetree* para copiar los ficheros de configuración necesarios a los contenedores LXC, incluyendo las claves públicas SSH autorizadas y los archivos de configuración de la base de datos. Estos contenedores tendrán que estar asociados a la VLAN correspondiente compartida con los *Pods* (servidores web y admin). Para ello, se puede partir del escenario VNX "k8s_lab-vms-vlan.xml", **utilizado al final de la práctica 1.3** y disponible entre los recursos del escenario de Kubernetes usado en el trabajo final, donde los contenedores LXC se conectan a la red *VlanNet* (red virtual compartida con los nodos *worker* de Kubernetes) para luego asociarlos a una VLAN concreta.
7. **Verificación de la conectividad:** comprobar que los *Pods* disponen de sus interfaces de red adicionales y que existe conectividad entre los servidores web, el servidor de administración y los contenedores LXC a través de la VLAN. Verificar también que el servidor de administración puede acceder mediante SSH a las máquinas correspondientes utilizando autenticación por clave pública/privada y que el servidor de administración y los servidores web pueden administrar la base de datos.

Cuando se necesite desplegar un recurso de Kubernetes que no disponga de un recurso específico en el proveedor de Terraform, puede analizarse el uso de *kubernetes_manifest*, que permite definir recursos a partir de su representación en HCL, incluidos recursos personalizados como los *NetworkAttachmentDefinition*. Para más información sobre el uso de *kubernetes_manifest* consulte la información disponible en [3].

## 3 - Personalización de páginas web de servidores Nginx

Se deberá modificar la configuración del despliegue de los servidores web para generar una página HTML personalizada a partir de una plantilla, sustituyendo un marcador por el nombre del *Pod* en el que se ejecuta el servidor web. Como orientación, se recomienda seguir los siguientes pasos:

1. ***ConfigMap* con la plantilla HTML:** crear un recurso *ConfigMap* que contenga el archivo *index.html* con la plantilla de la página web. Esta

deberá incluir un marcador que se sustituirá por el nombre del *Pod* en el que se ejecuta el servidor web Nginx.

2. **Generación de la página web:** implementar una de las siguientes alternativas. En ambos casos, el contenedor encargado de generar la página deberá leer la plantilla *index.html* del *ConfigMap*, sustituir el marcador por el nombre del *Pod* y escribir el resultado en un volumen compartido con el contenedor de Nginx.
    - **Opción A - Contenedor auxiliar (*sidecar*):** añadir un contenedor auxiliar al *Pod* que genere la página web. Este deberá obtener el nombre del *Pod* y generar el archivo index.html en un volumen compartido con el contenedor de Nginx.
    - **Opción B - *initContainer*:** añadir un contenedor de inicialización al *Pod* que genere el archivo *index.html* antes de que se inicie el contenedor del servidor web Nginx. Deberá sustituir el marcador de la plantilla por el nombre del *Pod* y escribir el resultado en el volumen compartido.
3. **Configuración de los volúmenes:** configurar los dos volúmenes necesarios para implementar la solución:
    - **Volumen del *ConfigMap*:** montar el *ConfigMap* para que el contenedor encargado de generar la página pueda acceder a la plantilla *index.html*. Este volumen será de solo lectura.
    - **Volumen compartido:** crear un volumen que permita escritura y montarlo tanto en el contenedor encargado de generar la página como en el contenedor de Nginx. El primero escribirá en él el archivo *index.html* generado y Nginx lo utilizará para servir la página web.

## 4 - Configuración de *Services* e *Ingress*

Se deberán definir los siguientes recursos de Kubernetes de tipo *Service* e *Ingress*:

1. ***Service* para los servidores web:** crear un *Service* de tipo ClusterIP que permita el acceso a los *Pods* de los servidores web desplegados mediante un *Deployment*. El Service expondrá el puerto TCP 8080 y lo redirigirá al puerto 80 de los *Pods*.
2. ***Service* para el servidor de administración:** crear un *Service* de tipo NodePort que permita acceder al *Pod admin* mediante SSH (TCP, puerto 22). El *Service* expondrá el puerto TCP 2222 y lo dirigirá al puerto 22 del *Pod*, utilizando el puerto *nodePort* asignado para el acceso desde el exterior del clúster. En el punto 3 del apartado 5 se comentan ajustes adicionales de este *Service* a tener en cuenta para la configuración de la *NetworkPolicy* asociada.
3. **Recurso *Ingress*:** definir un recurso *Ingress* que redirija las peticiones HTTP dirigidas a la ruta “/servers-GX” al *Service* asociado a los servidores web, sustituyendo “X” por el número del grupo de prácticas correspondiente.

## 5 - Configuración de recursos *NetworkPolicy*

Se deberán definir los siguientes tres tipos de políticas de red en Kubernetes mediante recursos de tipo *NetworkPolicy*:

1. **Política de denegación por defecto:** bloquear todo el tráfico de entrada (*Ingress*) y salida (*Egress*) de los *Pods* seleccionados.
2. **Política de acceso a los servidores web:** permitir el tráfico de entrada a los Pods que actúan como servidores web únicamente desde los *Pods* del *Ingress Controller* disponibles en el *namespace* "ingress-nginx". Para ello, se deberá utilizar *podSelector* para identificar los *Pods* autorizados mediante sus etiquetas (*labels*), que se podrán consultar con el comando "kubectl get pods --show-labels". Asimismo, se deberá especificar el *namespaceSelector* para indicar el *namespace* del *Ingress Controller*, mediante la etiqueta "kubernetes.io/metadata.name: ingress-nginx".
3. **Política de acceso al servidor de administración:** permitir el acceso SSH (TCP, puerto 22) al *Pod* del servidor de administración (*admin*) únicamente desde la IP del host autorizado (IP del host en red *user_LAN,* según la topología de red del escenario de Kubernetes representada en la Figura 2 de la práctica 1.4). Para ello, se utilizará *ipBlock* en lugar de *podSelector*, especificando la IP autorizada mediante su correspondiente CIDR.

   Recuerde que el acceso al *Pod admin* se realizará mediante un *Service* de tipo NodePort. Deberá tener configurada la opción "externalTrafficPolicy: Local" para preservar la IP origen del cliente (IP del host). Las conexiones SSH deberán dirigirse a la IP del nodo "k8s-worker1" y al puerto de acceso externo asignado al *Service*. Además, el *Pod* deberá incluir la opción *nodeSelector* para garantizar su despliegue en dicho nodo (**ver parte inicial de la práctica 1.3** para configurar la opción *nodeSelector*).

   Se deberá garantizar que únicamente se permita el acceso SSH al servidor *admin* desde la IP autorizada del host, teniendo en cuenta la conservación de la IP de origen durante el recorrido del tráfico.

Para más información sobre la configuración y el funcionamiento de los recursos *NetworkPolicy*, consultar la documentación oficial de Kubernetes [1], [2].

## Referencias


1 Terraform Kubernetes Provider.
https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs

2 Terraform Kubernetes Provider (GitHub Repository).
https://github.com/hashicorp/terraform-provider-kubernetes

3 Terraform Kubernetes Provider: kubernetes_manifest.
https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs/resources/manifest

4 Kubernetes Secrets. https://kubernetes.io/es/docs/concepts/configuration/secret

5 Terraform Cookbook. https://learning.oreilly.com/library/view/terraform-cookbook/9781098108458/

6 Terraform: Up and Running, 3rd Edition. https://learning.oreilly.com/library/view/terraform-up-and/9781098116736/

7 Terraform Kubernetes Provider: Manage & Deploy Resources. https://spacelift.io/blog/terraform-kubernetes-provider

8 Manage Kubernetes resources with Terraform. https://developer.hashicorp.com/terraform/tutorials/kubernetes/kubernetes-provider

9 How to Manage Kubernetes Resources with the Terraform Kubernetes Provider. https://oneuptime.com/blog/post/2026-02-20-terraform-kubernetes-provider/view

10 Using Terraform with Kubernetes: A Comprehensive Guide. https://medium.com/@vinoji2005/using-terraform-with-kubernetes-a-comprehensive-guide-237f6bbb0586